

Contents

1	リリース・デプロイ設計 (メインシステム)	1
1.1	0. 文書情報	1
1.2	1. リリース方針	1
1.2.1	1.1 リリース優先度の方針 (参照: 要件 §18.1)	1
1.2.2	1.2 機能ごとのリリース優先度 (参照: 要件 §18.2)	2
1.2.3	1.3 リリース判定基準 (参照: 要件 §18.3)	2
1.3	2. MVP リリース方式 (参照: 基本設計 §13.1)	2
1.3.1	2.1 MVP リリース判定期間	2
1.3.2	2.2 リリース判定 KPI (参照: 基本設計 §13.3)	2
1.3.3	2.3 シングルリージョン 99.9% 達成可能性の裏付け検証	2
1.4	3. 環境分離 (参照: 要件 §14.5)	3
1.5	4. リリースプロセス	3
1.6	5. デプロイ手順	4
1.6.1	5.1 Cloudflare Workers / Pages	4
1.6.2	5.2 シークレットローテーション	5
1.7	5a. オーナー固定化 + 論理削除統一 マイグレーション順序 (v1.10 履歴)	5
1.8	5b. D1/SQLite リファクタリング マイグレーション順序 (v1.10 履歴)	6
1.8.1	5b.1 マイグレーション順序	6
1.8.2	5b.2 既存データの不整合検出 SQL (M1~M3 開始前に実行)	6
1.8.3	5b.3 アプリ層の連動修正	7
1.8.4	5b.4 ロールバック方針	7
1.8.5	5b.5 想定リスクと緩和策	7
1.9	5c. v1.11 ユーザーマスタ 3 表分割マイグレーション (MVP 未リリース、DDL 切替のみ)	7
1.9.1	5c.1 マイグレーション順序 (forward-only、staging で検証 → prod 適用)	8
1.9.2	5c.2 「契約内メール一意」アプリ層担保	8
1.9.3	5c.3 v1.10 「オーナー自動 admin 行 INSERT」は維持	9
1.9.4	5c.4 PII 管理への影響	9
1.9.5	5c.5 ロールバック方針	9
1.9.6	5c.6 想定リスクと緩和策	9
1.10	6. ロールバック手順	9
1.10.1	6.1 ロールバック後の状態キャッシュ整合性検証 (runbook RB-008)	9
1.10.2	6.2 API v1 → v2 移行時のクライアント周知	10
1.11	7. 試験データ要件 (参照: 基本設計 §13.4)	11
1.11.1	7.1 試験データ生成スクリプト	11
1.12	8. 初期リリース	11
1.13	9. 災害復旧手順	11
1.14	10. 関連設計	11
1.15	11. 未確定事項・確認事項	12

1 リリース・デプロイ設計 (メインシステム)

1.1 0. 文書情報

項目	内容
文書名	リリース・デプロイ設計 (メインシステム)
運用 ID	OPS-05
対象システム	FAQ AI ウィジェット SaaS / メインシステム
作成日	2026-05-17
版数	v1.0
ステータス	承認済

1.2 1. リリース方針

1.2.1 1.1 リリース優先度の方針 (参照: 要件 §18.1)

優先度	方針
P0	初期リリース (MVP) に必須
P1	MVP の品質向上・運用性確保に必要
P2	MVP に含めるが、P0/P1 完了後に着手

1.2.2 1.2 機能ごとのリリース優先度 (参照: 要件 §18.2)

優先度	対象
P0	FAQ 管理(下書き含む)、FAQ 根拠限定回答、未解決質問登録、問い合わせ ID、個別チャット、メール登録、メール通知、未解決から FAQ 登録、処理エラー表示、課金基本機能、ユーザー管理、オーナー境界によるデータ分離、HTTPS・レート制限・操作ログ
P1	FAQ カテゴリ、FAQ 候補の下書き支援、FAQ 登録元参照、分析、対応テンプレート、ウィジェットテーマ設定、アクセシビリティ強化、データエクスポート、データリージョン選択、AI モデル切替

1.2.3 1.3 リリース判定基準 (参照: 要件 §18.3)

観点	判定基準
機能	全 P0 機能が実装され、受入条件 AC-001~AC-044(分解された AC-040a~c / AC-044a~b を含む)・AC-046・AC-047 を満たす
性能	AC-046 のとおり、MVP リリース判定期間で NFR-101~106 の p95 目標を 4 週間連続達成
セキュリティ	オーナー境界によるデータ分離、ドメイン検証、レート制限、HTTPS が動作する
プライバシー	利用目的表示が用意されている
運用	監視・アラート・障害対応手順が整備されている
ドキュメント	利用ガイド・運用手順書が整備されている

1.3 2. MVP リリース方式 (参照: 基本設計 §13.1)

MVP はリリース判定期間を設け、招待制 10 契約で稼働確認を行う。

1.3.1 2.1 MVP リリース判定期間

段階	期間	対象	判定 KPI
MVP リリース判定期間	4 週間	本番環境の招待オーナー / 招待契約	§ 2.2 参照

1.3.2 2.2 リリース判定 KPI(参照: 基本設計 §13.3)

全件達成必須:

KPI	内容
(a) 可用性目標達成	NFR-201~204 を達成
(b) P1 重大障害ゼロ	MVP リリース判定期間中
(c) オーナー境界によるデータ分離自動テスト	100% 合格(要件 § 14.3)
(d) AC-001~022 全件合格	受入条件(付録 C)
(e) サポート問い合わせ平均 1 次応答	1 営業日以内
(f) AC-046 性能 4 週間連続達成	NFR-101~106 p95 を MVP リリース判定期間で 4 週間連続達成
(g) AC-047 アクセシビリティ達成	WCAG 2.1 AA 自動 + 手動評価
(h) シングルリージョンで NFR-201 99.9% 達成可能性の裏付け検証	apac 単一リージョンで運用し、MVP リリース判定期間の実測で SLO 達成可能性を確認

1.3.3 2.3 シングルリージョン 99.9% 達成可能性の裏付け検証

本サービスは **apac 単一リージョン固定** で、データ越境ゼロ (APPI / NFR-805) を担保している。SLO 目標は公開ウィジェット 99.9%(NFR-201)・個別チャット 99.9%(NFR-203)。単一リージョン構成で 99.9%(年間ダウンタイム 8.76 時間以内) を達成できる根拠を以下で検証する。

項目	確認方法	合格基準
(1) Cloudflare apac リージョン過去 12 ヶ月実績	Cloudflare Status Page アーカイブ + 公式 SLA レポート	apac 主要サービス (Workers / D1 / R2 / KV / Queues) の月次稼働率 ☑ 99.95% を 11 ヶ月以上
(2) 本サービス MVP リリース判定期間中の月次稼働率	Cloudflare Workers Analytics Engine + SCR-096 KPI ダッシュボード	4 週間連続で NFR-201 / NFR-203 を 99.9% 以上維持
(3) RTO/RPO 達成見込み (DR シナリオ別)	バックアップ・復旧 + 四半期 DR 訓練実績 (NFR-803)	各シナリオで RTO 4 時間 / RPO < 1 時間達成
(4) Cloudflare サービス全断時のフォールバック	AI しきい値 3 段フォールバック / 縮退運転 (顧客管理)	Cloudflare 部分障害でもエンドユーザー側ウィジェット最低機能が継続可能
(5) SLO 違反予算 (エラーバジェット) 計算	burn-rate 二重監視 (1h / 24h)	MVP リリース判定期間の累積ダウタイムが年間バジェット 8.76 時間に対して 0.73 時間/月以下を維持

DR シナリオ別 RTO/RPO 分解表

シナリオ	想定インパクト	RTO 内訳	RPO 内訳	合格基準
(3-a) D1 単一インスタンス障害	全契約書込不可	検知 5 分 + 自動フェイルオーバー 30 分 + 整合性検証 1 時間 = 約 1.6 時間	D1 Time Travel = 15 分以内	RTO 4h / RPO < 1h ⇒ 達成
(3-b) D1 + R2 同時障害 (apac リージョン主要サービス全断)	全機能停止	検知 5 分 + R2 復元 2 時間 + D1 復元 1 時間 + 検証 30 分 = 約 3.5 時間	R2 日次 = 最大 24 時間、Time Travel 併用で 15 分	RTO 4h / RPO 通常時 < 1h
(3-c) Workers Runtime グローバル障害	API / ウィジェット停止	Cloudflare 復旧待ち (過去事例 < 1 時間) + 自動復旧	0	RTO 1h / RPO 0 ⇒ 達成
(3-d) Queues / KV 部分障害	通知遅延、キャッシュミス	DLQ 30 日 R2 退避 + KV TTL 失効後 D1 永続フォールバック	0 ~ 30 分	RTO 4h / RPO < 1h ⇒ 達成

(1)~(5) の合格基準を全て満たした場合、PO + SRE + 経営の 3 名承認で「シングルリージョンで 99.9% 達成可能」と判定。

1.4 3. 環境分離 (参照: 要件 §14.5)

環境	Workers / D1 / R2 / KV	データ	デプロイ権限
dev	開発者個別	合成データ	全開発者
staging	共有	本番マスキング済みダンプ (週次更新)	開発者
prod	共有	本番	2 名承認 (GitHub OIDC + Environment Protection Rules)

- ・ **本番データのマスキング**: メールアドレスは SHA-256 ハッシュ化、氏名・住所はダミー値、本文は先頭 100 文字 + … でトリミング
- ・ **本番直接変更**: 緊急区分発動時に限り、対応チケット ID と監査ログ記録を必須 (`db.query.execute action` コード)
- ・ **マイグレーション**: スキーマバージョン管理を必須とし、ダウンタイム無しのスキーマ変更は **expand/contract 二段階** (まず追加、移行後に削除) で実施
- ・ **デプロイ権限**: 本番への書き込み権限 (Workers デプロイ、D1 マイグレーション、R2 オブジェクト変更) を持つアカウントを最小化し、定期棚卸しを実施
- ・ **DB 操作者ロール**: NFR-319 補完策に従い 2 名以下に限定、月次棚卸しで PO 承認

1.5 4. リリースプロセス

段階	内容
開発 (dev)	feature branch + PR、開発者個別の dev 環境 (合成データ)
レビュー	コードレビュー、テスト追加、テスト戦略に従う
ステージング (staging)	E2E、回帰テスト、本番マスキング済みダンプ週次更新
カナリア	一部トラフィックでリリース
本番展開 (prod)	2 名承認 (GitHub OIDC + Environment Protection Rules)
ロールバック	不具合検知時に直前バージョンへ戻す
本番直接変更	緊急時のみ運営者の対応チケット ID 必須、全クエリを監査ログに記録 (db.query.execute)
マイグレーション	expand/contract 二段階、ダウンタイムゼロ (R-013)

1.6 5. デプロイ手順

1.6.1 5.1 Cloudflare Workers / Pages

1.6.1.1 5.1.1 Wrangler 典型コマンドフロー D1 マイグレーション新規作成から本番反映までの典型フロー。forward-only 原則を遵守。

(1) マイグレーションファイル作成 (連番で命名、シャープ禁止)

```
wrangler d1 migrations create main-db "0042_add_users_timezone_column"
```

→ migrations/0042_add_users_timezone_column.sql が生成される

(2) ローカル D1 で適用 + 単体テスト

```
wrangler d1 migrations apply main-db --local
pnpm --filter '@faq-saas/main-api' test
```

(3) staging 適用

```
wrangler d1 migrations apply main-db-staging --remote
pnpm test:integration -- --env=staging
```

(4) 本番適用 (PR マージ後 / CI 経由を推奨。手動時は 4-eyes)

```
wrangler d1 migrations apply main-db-prod --remote
```

(5) 適用済確認 (適用済 migration 一覧)

```
wrangler d1 migrations list main-db-prod --remote
```

(6) 単発 SQL 実行 (緊急時のみ。原則 migration ファイル経由)

```
wrangler d1 execute main-db-prod --remote --command="ANALYZE audit_logs;"
```

(7) ロールバック (forward-only のため、新規 migration で前進修正)

```
wrangler d1 migrations create main-db "0043_rollback_users_timezone"
```

→ 0043 で前進修正の SQL を書き、(1)~(4) を繰り返す

1.6.1.2 5.1.2 通常デプロイ手順

1. D1 マイグレーション (forward-only)

```
wrangler d1 migrations apply main-db-prod --remote
```

2. Workers デプロイ (順序: shared → consumer/cron → API)

```
pnpm --filter '@faq-saas/shared' build
pnpm --filter '@faq-saas/queue-consumer' build && cd app/workers/queue-consumer && wrangler deploy
pnpm --filter '@faq-saas/cron' build && cd app/workers/cron && wrangler deploy --env prod
pnpm --filter '@faq-saas/main-api' build && cd app/workers/main-api && wrangler deploy --env prod
pnpm --filter '@faq-saas/widget-api' build && cd app/workers/widget-api && wrangler deploy --env prod
pnpm --filter '@faq-saas/internal-api' build && cd app/workers/internal-api && wrangler deploy --env prod
pnpm --filter '@faq-saas/webhook' build && cd app/workers/webhook && wrangler deploy --env prod
```

3. Pages デプロイ

```
pnpm --filter '@faq-saas/admin' build && cd app/admin && wrangler pages deploy dist --project-name admin
pnpm --filter '@faq-saas/widget' build && cd app/widget && wrangler pages deploy dist --project-name widget
pnpm --filter '@faq-saas/public' build && cd app/public && wrangler pages deploy dist --project-name public
```

4. スモークテスト

```
pnpm test:smoke -- --env=prod
```

1.6.2 5.2 シークレットローテーション

Master Key 年次ローテーション

```
wrangler secret put MASTER_KEY_NEXT --env prod
```

→ アプリ側のデュアル読み取り版をデプロイ

→ バックフィルジョブで再暗号化

→ **MASTER_KEY** を新キーに上書き、旧キーを **MASTER_KEY_PREV** に

→ 1 年後 **MASTER_KEY_PREV** 削除

1.7 5a. オーナー固定化 + 論理削除統一 マイグレーション順序 (v1.10 履歴)

注記: 本セクション (M1~M10、§ 5a / § 5b) は v1.10 リファクタ時点の手順記録 (履歴) である。MVP 未リリースのため、その後の v1.11 でユーザー情報管理を 3 表分割 (**users** / **contract_owners** / **project_users**) へ再リファクタリングしたため、本セクション内の **accounts** / **account_project_grants** を直接参照する SQL や手順は新スキーマでは適用できない。新規環境への適用手順は § 5c 「v1.11 ユーザーマスタ 3 表分割マイグレーション」を参照する。本セクションは設計判断の文脈保持 (なぜ valid カラムを全テーブルに付けたか、なぜ削除動線が SCR-026 集約になったか等) のため残置する。

本変更を本番反映する際の手順 (ゼロダウンタイム前提):

1. DDL マイグレーション (forward-only):

- 11 テーブル (**accounts** / **account_project_grants** / **projects** / **allowed_domains** / **project_ip_allowlist** / **faqs** / **question_logs** / **inquiries** / **inquiry_contacts** / **chat_rooms** / **billing_subscriptions**) に valid INTEGER NOT NULL DEFAULT 1 CHECK (valid IN (0,1)) カラムを追加
- 既存行は valid=1 がデフォルトで自動セットされるため backfill 不要

2. インデックス再作成:

- accounts.uq_accounts_owner_email** を DROP → CREATE UNIQUE INDEX **uq_accounts_owner_email** ON **accounts**(owner_account_id, email_hmac) WHERE valid = 1 で部分 UNIQUE 化
- 各テーブルに **idx_<table>_valid** ON <table>(valid) WHERE valid = 0 を追加

3. 既存オーナー × 全プロジェクトのバックフィル INSERT:

- 既存契約に対し INSERT INTO **account_project_grants**(account_id, project_id, role, valid, granted_by, granted_at) SELECT p.owner_account_id, p.id, 'admin', 1, p.owner_account_id, datetime('now') FROM **projects** p WHERE NOT EXISTS (SELECT 1 FROM **account_project_grants** apg WHERE apg.account_id=p.owner_account_id AND apg.project_id=p.id) を実行
- 監査ログ **project.backfill_owner_admin_v1_10** を retention_class='general' で発火

4. アプリ層クエリ切替:

- 全 GET 系エンドポイントのクエリに WHERE <table>.valid=1 フィルタを追加 (段階的にデプロイ可能、valid=1 は新規行のデフォルトのため後方互換)
- 認証モドルウェアに **accounts.valid=1** チェック追加 (valid=0 行は 401 でログイン拒否)

5. DELETE 系 API を論理削除に切替:

- DELETE /projects/{id}/members/{accountId} を物理削除 → 論理削除 (UPDATE **accounts** SET valid=0) に変更
- DELETE /projects/{id} を物理削除 → 論理削除 (UPDATE **projects** SET status='deleted', valid=0) に変更

6. UI 切替:

- SCR-010 / SCR-010-M1 から削除ボタン撤去
- SCR-010-M1 から「このプロジェクトの管理者を指定」セクション撤去
- SCR-026 にプロジェクト削除動線を追加 (L3 = TypeToConfirmInput + ReauthConfirmModal)
- SCR-017-M1 確認文言を物理 → 論理削除セマンティクスに変更

7. 90 日後物理削除バッチ有効化:

- DD14 § 3.1.11 のバッチを cron(0 18 * * *) (JST 03:00) で有効化
- 初回起動は本マイグレーションのデプロイから 90 日後 (早すぎる物理削除を回避)

ロールバック手順: アプリレベルは wrangler rollback、DB スキーマは D1 Time Travel で本マイグレーション実行前へ巻き戻し可能。**account_project_grants** のオーナー admin 行が大量にバックフィルされている場合の DELETE 戦略は別途検討 (運営者経由のクリーンアップ)。

1.8 5b. D1/SQLite リファクタリング マイグレーション順序 (v1.10 履歴)

本変更を本番反映する際の手順(計画停止を最小化するため、再作成系は運用窓を確保):

D1 制約: ALTER TABLE での CHECK 制約変更 / FK 制約追加は不可。該当箇所は「新テーブル作成 → INSERT … SELECT → DROP TABLE → ALTER TABLE RENAME TO」方式を用いる。**PRAGMA foreign_keys = OFF** で外部キーを一時無効化し、最後に **PRAGMA foreign_key_check** で整合確認 → ON に戻す。

1.8.1 5b.1 マイグレーション順序

ステップ	内容	方式	リスク
M1	announcement_audiences / chat_room_reminder_history 新規 CREATE TABLE + インデックス	DDL のみ	低
M2	既存 service_announcements.audience_owner_account_ids(JSON 配列) → announcement_audiences データ移行 (json_each で行展開)	データコピー	中 (JSON 非互換行の検出)
M3	service_announcements.audience_owner_account_ids を DROP COLUMN、created_by に REFERENCES accounts(id) ON DELETE SET NULL を付与するためテーブル再作成	テーブル再作成	中
M4	accounts テーブル再作成 (deleted_at 列削除 + email_key_version 列追加 + role CHECK から service_operator 削除)	テーブル再作成	高 (参照先が広範)
M5	projects / faqs / inquiries の deleted_at を DROP COLUMN (D1 / SQLite 3.35+ サポート)	ALTER TABLE	低
M6	inquiry_contacts.email_key_version を ADD COLUMN	ALTER TABLE	低
M7	audit_logs テーブル再作成 (prev_hash / current_hash / tombstone 列削除、FK 明示)。tombstone=1 行は捨象 (運営者側に正本あり)	テーブル再作成	中
M8	notification_logs テーブル再作成 (owner_account_id / inquiry_id / chat_message_id の FK 明示)	テーブル再作成	中
M9	account_project_grants テーブル再作成 (granted_by の NOT NULL 解除 + ON DELETE SET NULL 付与)	テーブル再作成	中
M10	ai_threshold_persistent_cache テーブル再作成 (scope と owner/project の整合 CHECK 追加 + しきい値範囲 CHECK + FK 明示)	テーブル再作成	低
M11	faq_search_fts 同期トリガを CREATE (既存なら幂等にスキップ)	CREATE TRIGGER	低
M12	inquiry_status_history.changed_by / question_logs 系の FK 明示のためテーブル再作成 (必要箇所のみ)	テーブル再作成	低
M13	chat_messages.actor_id / faqs.created_by, updated_by, source_unresolved_question_id の FK / ON DELETE 句明示のためテーブル再作成	テーブル再作成	低
M14	bash 99_script/check-spec-sync.sh 実行	スクリプト	—

1.8.2 5b.2 既存データの不整合検出 SQL (M1～M3 開始前に実行)

-- 1. deleted_at 非 NULLなのに valid=1 の不整合

```
SELECT 'accounts' AS tbl, id FROM accounts WHERE deleted_at IS NOT NULL AND valid = 1
UNION ALL SELECT 'projects', id FROM projects WHERE deleted_at IS NOT NULL AND valid = 1
UNION ALL SELECT 'faqs', id FROM faqs WHERE deleted_at IS NOT NULL AND valid = 1
UNION ALL SELECT 'inquiries', id FROM inquiries WHERE deleted_at IS NOT NULL AND valid = 1;
```

-- 2. accounts.role = 'service_operator' の混入チェック

```
SELECT id, email_hmac FROM accounts WHERE role = 'service_operator';
```

-- 3. service_announcements.audience_owner_account_ids が JSON 非互換

```
SELECT id FROM service_announcements
WHERE audience_owner_account_ids IS NOT NULL
AND json_valid(audience_owner_account_ids) = 0;
```

-- 4. account_project_grants.granted_by の参照切れ

```
SELECT apg.account_id, apg.project_id, apg.granted_by
FROM account_project_grants apg
LEFT JOIN accounts a ON a.id = apg.granted_by
WHERE apg.granted_by IS NOT NULL AND a.id IS NULL;
```

-- 5. ai_threshold_persistent_cache の scope/owner/project 不整合

```
SELECT id, scope, owner_account_id, project_id
FROM ai_threshold_persistent_cache
WHERE NOT (
    (scope = 'global' AND owner_account_id IS NULL AND project_id IS NULL)
    OR (scope = 'owner' AND owner_account_id IS NOT NULL AND project_id IS NULL)
    OR (scope = 'project' AND owner_account_id IS NOT NULL AND project_id IS NOT NULL)
);
```

1.8.3 5b.3 アプリ層の連動修正

- **accounts.role CHECK 変更:** 認可ミドルウェアで service_operator 値を許容している箇所を削除 (運営者は別 D1)
- **お知らせ作成:** service_announcements 単独 INSERT から、service_announcements + announcement_audiences の 2 段 INSERT (D1 batch API) に変更
- **配信先取得 API:** audience_owner_account_ids の JSON パースから JOIN announcement_audiences に切替
- **論理削除日時参照:** deleted_at を見ていたコードを updated_at に統一
- **暗号鍵バージョン:** 復号処理で email_key_version 列を読み、対応する を Secrets Store から取得 (セキュリティ設計 § 8.2.1)
- **audit_logs:** メイン側で prev_hash / current_hash を計算していたコードを削除し、IF #8 で運営者側に転送する形に統一
- **chat_rooms.reminder_state 更新:** 履歴行の挿入を D1 batch API で同時実行
- **DD14 物理削除バッチ:** § 3.1.7 / § 3.1.8 / § 3.1.9 / § 3.1.11 のクエリで deleted_at 参照を valid / updated_at に置換 (本マイグレーション併走で実施)

1.8.4 5b.4 ロールバック方針

- 新規テーブル (announcement_audiences / chat_room_reminder_history) は DROP TABLE
- DROP COLUMN は同名で ALTER TABLE ADD COLUMN 再追加 (値は復元不能、運用許容判断 —— deleted_at は updated_at で代替できるため実害なし)
- テーブル再作成系は事前に <table>_v110_backup 接尾辞でバックアップを残置し、運用窓終了で DROP
- アプリレベルは wrangler rollback、DB スキーマは D1 Time Travel で巻き戻し (最大 30 日)

1.8.5 5b.5 想定リスクと緩和策

リスク	影響	緩和策
accounts テーブル再作成中の参照整合性違反	高	PRAGMA foreign_keys=OFF でマイグレーション、終了時 foreign_key_check
service_announcements.audience_owner_account_ids が JSON 非互換形式	中	M2 開始前に § 5b.2 SQL #3 で全件 json_valid チェック、非互換行は手動で配列化
audit_logs 再作成時に運営者側 IF #8 同期がズれる	中	マイグレーション窓に IF #8 同期を一時停止し、終了後再開。再開時に再送防止のため Idempotency-Key を厳格化
D1 のマイグレーションタイムアウト (30 秒上限)	中	大規模テーブルは分割マイグレーション (例: audit_logs を created_at 範囲ごとに ETL)
email_key_version=1 既定値で dual-decrypt が動かない	低	アプリ側で「version 未指定 = 1」フォールバック実装

1.9 5c. v1.11 ユーザーマスタ 3 表分割マイグレーション (MVP 未リリース、DDL 切替のみ)

前提: MVP 未リリースのためデータ移行不要。新規 DDL のみ。旧 v1.10 の accounts / account_project_grants 単一表モデルを、関心分離の明示と DB 制約強化のために **3 表分離型** (users / contract_owners / project_users) へ切り替える。設計プランは </Users/sasakikouhei/claude/plans/humble-yawning-beacon.md> を正本、新スキーマの DDL は [./02_基本設計/03_テーブル設計.md](#) § 3.1 / § 3.1a / § 3.2 を正本とする。

1.9.1 5c.1 マイグレーション順序 (forward-only、staging で検証 → prod 適用)

ステップ	内容	方式	リスク
M11-1	旧 accounts / account_project_grants の DDL を撤去 (DROP TABLE。 MVP 未リリースのため依存データなし)	DDL のみ	低
M11-2	新 DDL を適用 (基本設計 03_ テーブル設計.md § 3.1 / § 3.1a / § 3.2 を正本): ・ users(ユーザーマスタ、13 カラム。全ロール共通の認証情報) ・ contract_owners(契約者ユーザー、user_id PRIMARY KEY で 1 契約 = 1 オーナー固定。contract_status / data_deletion_mode 専有) ・ project_users(プロジェクトユーザー割当、(user_id, project_id) 複合 PK + contract_owner_user_id 冗長保持)	CREATE TABLE	低
M11-3	9 テーブルの FK 書換 + カラムリネーム (契約境界系 → contract_owners(user_id)、操作者系 → users(id)) ・ 契約境界 (CASCADE): projects.contract_owner_user_id / faqs.contract_owner_user_id / inquiries.contract_owner_user_id / question_logs.contract_owner_user_id / billing_subscriptions.contract_owner_user_id / billing_invoices.contract_owner_user_id / usage_metering.contract_owner_user_id / owner_quota_overrides.contract_owner_user_id / audit_logs.contract_owner_user_id / notification_logs.contract_owner_user_id / inbox_messages.contract_owner_user_id / withdrawal_requests.contract_owner_user_id / ai_threshold_persistent_cache.contract_owner_user_id / error_logs.contract_owner_user_id ・ 操作者 (SET NULL): faqs.created_by / faqs.updated_by / inquiries.assignee_user_id / chat_messages.actor_id / sessions.user_id / access_tokens.user_id / terms_agreements.user_id / project_users.granted_by / audit_logs.actor_user_id / error_logs.user_id	テーブル再作成	高 (参照先広範)
M11-4	インデックス再作成 (基本設計 03_ テーブル設計.md 各節の CREATE INDEX を反映): ・ idx_users_email_hmac / idx_users_valid ・ idx_contract_owners_status / idx_contract_owners_valid ・ idx_project_users_project / idx_project_users_owner / idx_project_users_role_admin / idx_project_users_valid ・ uq_usage_metering_owner_month(contract_owner_user_id, billing_year_month) ・ uq_billing_invoices_owner_month(contract_owner_user_id, billing_year_month) ・ idx_faqs_owner_project / idx_inquiries_contract_status_created / idx_qlog_owner_created 等の contract_owner_user_id 系インデックス	CREATE INDEX	低
M11-5	seed / E2E fixtures (scripts/seed-staging.ts 等) を新 3 表 INSERT に変更: ・ users を先に INSERT ・ オーナーは contract_owners(user_id=...) を追加 INSERT (行存在 = オーナー判定) ・ プロジェクト作成と同時に project_users(user_id= オーナー, project_id, contract_owner_user_id= オーナー, role='admin', valid=1) を自動 INSERT	スクリプト改修	低
M11-6	アプリ層クエリの一括書換 (account_id → user_id、owner_account_id → contract_owner_user_id、accounts → 文脈で users / contract_owners 振り分け、assignee_account_id → assignee_user_id、actor_account_id → actor_user_id、accountId → userId、ownerAccountId → contractOwnerUserId、is_owner フラグ参照 → contract_owners 行存在判定に置換)	アプリ改修	中
M11-7	bash 99_script/check-spec-sync.sh で SC-001~SC-011 グリーン確認	スクリプト	—

1.9.2 5c.2 「契約内メール一意」アプリ層担保

旧 uq_accounts_owner_email(owner_account_id, email_hmac) WHERE valid=1 は新スキーマ (users 単独) では表現不能。アプリ層トランザクションで担保する (./02_基本設計/03_テーブル設計.md § 3.1 設計判断、§ 3.2 メンバー招待時手順を正本):

- ・ **オーナー登録時**: 単に新 users 行 INSERT (契約はまだ無いので重複なし)
- ・ **メンバー招待時**: BEGIN IMMEDIATE で「同一 contract_owner_user_id 配下の既存 project_users.user_id の users.email_hmac セット」+「contract_owners 行の users.email_hmac」を SELECT し重複検査 → 新 users 行 INSERT → project_users 行 INSERT

- ・ テストケース: `it-contract-email-unique-001`(同時招待 2 リクエストで片方が `E-BIZ-DUPLICATE-EMAIL` で弾かれること)

1.9.3 5c.3 v1.10 「オーナー自動 admin 行 INSERT」は維持

プロジェクト作成 API は引き続き同一トランザクション内で `INSERT INTO project_users(user_id=actor, project_id=newId, contract_owner_user_id=actor, role='admin', valid=1, granted_at=now())` を 1 行発行する。認可は `isOwner` `bypass(contract_owners` 行存在で判定) を最優先、`project_users` 行は critical 通知宛先解決と画面の「プロジェクト管理者一覧」表示のための事実情報として参照する。

1.9.4 5c.4 PII 鍵管理への影響

`deriveOwnerKey(masterKey, ownerId)` の引数 `ownerId` セマンティクスを「`contract_owners.user_id`(= 旧 `accounts.id` と物理値同じ ULID)」に再定義。DD11 §10.9.2 の HKDF info 文字列 `owner:<id>` の中身は不変。MVP 未リリースのためバックフィル不要。ローテーション単位は引き続き「契約 1 つ = `contract_owners` 1 行」。

1.9.5 5c.5 ロールバック方針

- ・ 新 3 表 DDL を DROP TABLE → 旧 `accounts / account_project_grants` DDL を再適用 (v1.10 §5a / §5b 履歴を参照)
- ・ アプリレベルは `wrangler rollback`、DB スキーマは D1 Time Travel で巻き戻し (MVP 未リリースなので運用窓制約は緩い)
- ・ 12 テーブル論理削除リスト ([CLAUDE.md](#) 同期ルール §) も同期して v1.10 の 11 テーブル (`accounts / account_project_grants / …`) に戻すこと

1.9.6 5c.6 想定リスクと緩和策

リスク	影響	緩和策
<code>project_users.contract_owner_user_id</code> と <code>projects.contract_owner_user_id</code> の不整合	高 (認可境界クエリの結果が齟齬する)	アプリ層 INSERT で必ず両者一致を担保、レビューチェックリスト化、将来トリガで強制検討
<code>is_owner</code> フラグ参照の sed 漏れ	中 (認可ロジック退行)	<code>check-spec-sync.sh</code> SC-002 / SC-007 の検査キーワード差替で機械検出、PR レビュー必須
404 偽装ロジック (<code>principal.contractOwnerId !== resource.contract_owner_user_id</code>) の比較箇所 sed 漏れ	高 (クロス契約データ漏洩)	一括 <code>grep</code> + 単体テストで対象 9 テーブルの 404 偽装をすべて回帰
旧 <code>uq_accounts_owner_email</code> 相当のアプリ層担保漏れ	中 (同契約内重複メール混入)	<code>it-contract-email-unique-001</code> 必須通過、レビューチェックリスト

1.10 6. ロールバック手順

マイグレーションは forward-only のため、ロールバックは新規 migration で前進修正。

- ・ アプリレベル: `wrangler rollback` で直前のデプロイにロールバック可能 (DB スキーマと不整合な場合は手動修正)
- ・ DB レベル: D1 Time Travel で巻き戻し (最大 30 日)

1.10.1 6.1 ロールバック後の状態キャッシュ整合性検証 (runbook RB-008)

`wrangler rollback` はアプリコードを巻き戻すが、KV / R2 はロールバックされない。アプリの新版で書き込んだ KV キーや R2 オブジェクトが旧版で解釈できないと、キャッシュ汚染やデータ参照不整合が発生する。

領域	ロールバック影響	検証手順
KV 共通キャッシュ (<code>project_key:* / ai-params:*</code> 等、TTL 60s 系)	TTL 60s で自然失効するので大局影響なし。ただしロールバック直後 60s は新版で書き込んだ値を旧版が読む可能性	ロールバック直後に <code>wrangler kv key delete --binding=KV_CACHE --bulk</code> で関連プレフィックスを明示削除 (RB-008 §A)

領域	ロールバック影響	検証手順
KV 設定キー (feature:*/ai-cost:unit-prices 等、永続)	旧版でフィールドが認識できない場合は default fallback で動作。新フィールド追加なら影響なし、削除なら旧版が undefined を null 扱いするか確認	RB-008 §B チェックリストで旧版互換性確認
KV 冪等性ストア (idempotency:*/webhook:idempotency:*)	24 時間～30 日の TTL で残存、redeploy 後の同一キーで前回結果を返してしまうリスク	rollback 直後 24h 以内に冪等性キャッシュをクリア (RB-008 §C)
R2 オブジェクト (audit-archive:*/backups:*/dlq/*)	永続。旧版で書き込み形式が異なると整合性破綻	R2 lifecycle で新版オブジェクトを識別、ロールバック後は新版書込済オブジェクトを R2 から手動移動
D1 schema	Time Travel で 30 日以内に巻き戻し可能。ただし KV / R2 の整合性は別途確認	wrangler d1 time-travel restore main-db-prod --timestamp=<rollback 時刻>

1.10.1.1 6.1.1 ロールバック手順 (高レベル)

1. アプリロールバック

```
wrangler rollback --env prod # 直前デプロイへ
```

2. DB schema 影響あれば D1 Time Travel

```
wrangler d1 time-travel info main-db-prod
```

```
wrangler d1 time-travel restore main-db-prod --timestamp=2026-05-13T10:00:00Z
```

3. KV キャッシュクリア (RB-008 §A 関連プレフィックス)

```
wrangler kv key list --binding=KV_CACHE --prefix=project_key: | xargs -I {} wrangler kv key del
```

4. 冪等性キャッシュクリア

```
wrangler kv key list --binding=KV_CACHE --prefix=idempotency: | xargs ...
```

5. スモークテスト

```
pnpm test:smoke -- --env=prod
```

6. 監査ログ記録 (4-eyes)

```
# → audit_logs に prod.rollback (5y 保持) を記録
```

1.10.1.2 6.1.2 必須事後検証

- D1 スキーマ ☒ アプリコード版整合性 (wrangler d1 migrations list の最終 migration がアプリ要求バージョン以下か)
- KV キーカウント (wrangler kv key list --binding=KV_CACHE | wc -l) のロールバック前後差分
- R2 オブジェクトリスト (wrangler r2 object list <bucket>) の不整合チェック
- スモークテスト全 PASS

1.10.2 6.2 API v1 → v2 移行時のクライアント周知

v2(breaking change) リリース時の **ウィジェット埋込先契約への周知** プロトコル:

タイミング	チャネル	内容
60 日前	開発者ブログ (https://docs.example.com/changelog/v2-migration)	概要 + 移行ガイド公開
30 日前	オーナー / メンバー (ユーザー管理権限保持)宛 in-app 通知 (重要度 high)	期限 + 移行リンク + 影響範囲
30 日前	メール (オーナー (オーナーアカウント)宛、Resend 送信)	同上
7 日前	オーナー / メンバー (ユーザー管理権限保持)宛 in-app 通知 (critical) + メール	最終警告
廃止日前日	オーナー / メンバー (ユーザー管理権限保持)宛 in-app 通知 (critical) + メール + ウィジェット側の sentry 風バナー (本番 v1 アクセス時に「24h 後に廃止」表示)	翌日廃止
廃止日	v1 エンドポイントを 410 GONE で返却	Sunset / Link ヘッダで v2 のドキュメント URL を提示

タイミング	チャンネル	内容
廃止後 30 日	v1 ルーティング自体を Worker から削除	完全撤去

- ・ **テスト**: 移行期間中の v1/v2 並走を **it-api-v1v2-coexist-001** で検証 (同一エンドポイントへの v1/v2 リクエストが独立した response schema で返ることを確認)
- ・ **メトリクス**: v1 利用率を **sli_api_v1_usage_rate** で監視 (廃止 7 日前で 10% 未満になっていないとリスク判断)

1.11 7. 試験データ要件 (参照: 基本設計 §13.4)

項目	件数 (MVP)	備考
検証用契約	5 件	用途別: (a) 通常運用、(b) サスペンション中、(c) 削除予定、(d) トライアル中、(e) 上限近傍
検証用プロジェクト	契約あたり 3~5 件	アーカイブ済み 1 件含む
検証用 FAQ	プロジェクトあたり 50 件	下書き / 公開 / 非公開 / カテゴリ付与のミックス
想定質問セット	50 件以上	AI 品質回帰テスト用 (FR-059)
エンドユーザー疑似データ	1 契約あたり 100 件	メール・問い合わせ ID・チャットメッセージ
検証用運営者アカウント	3 名	役割: (a) 全権、(b) 課金 / 契約管理のみ、(c) 監査ログ閲覧のみ
プロンプト注入攻撃セット	20 ケース以上	NFR-310 / FR-058 / AC-035 と整合

生成方法: 試験データ生成スクリプトを基本設計工程で整備する。生成スクリプトは個人情報保護のため擬似データ専用 (faker 等) を採用し、本番由来データは使用しない。

1.11.1 7.1 試験データ生成スクリプト

```
// scripts/seed-staging.ts
import { D1Database } from '@cloudflare/workers-types';
async function seed(db: D1Database) {
  const contractOwnerUserId = 'user_test_owner_001';
  // 1. users (全ロール共通の認証主体)
  await db.prepare(`INSERT INTO users (id, ...) VALUES (?1, ...)`)
    .bind(contractOwnerUserId, ..
  // 2. contract_owners (行存在 = オーナー判定。`user_id` PK で 1 契約 1 オーナー固定)
  await db.prepare(`INSERT INTO contract_owners (user_id, contract_status, data_deletion_mode,
  // 3. project_users (プロジェクト作成と同時にオーナー自身の admin 行を自動 INSERT)
  for (let i = 0; i < 100; i++) {
    await db.prepare(`INSERT INTO faqs (id, contract_owner_user_id, project_id, ...) VALUES (?1
  }
  // ... 全テーブルにテストデータ投入
}
```

実行: `pnpm tsx scripts/seed-staging.ts --env staging`

1.12 8. 初期リリース

移行データなし (新規 SaaS)。テスト用シードデータのみ。

1.13 9. 災害復旧手順

[02_バックアップ・リストア設計.md](#) を参照。

1.14 10. 関連設計

種別	参照先
要件	../01_ 要件定義/index.md § 14.5 / § 18
基本設計	../02_ 基本設計/index.md § 13 リリース戦略
詳細設計	../03_ 詳細設計/index.md § 18 移行・リリース設計
関連 NFR	NFR-201 / NFR-202 / NFR-203 / NFR-802 / NFR-803 / R-013 / AC-046 / AC-047

1.15 11. 未確定事項・確認事項

確認事項 ID	確認内容	優先度	ステータス
-	v1.0 リリース時点で全項目確定済み	低	確認済